



Proyecto 1

Reporte técnico de la aplicación: Chat



Francisco Uriel Pineda Hernández

`ciscohdz@ciencias.unam.mx`

Facultad de Ciencias

Universidad Nacional Autónoma de México

Fecha de entrega: 28 de septiembre de 2026

Resumen

Se describe el diseño e implementación de una aplicación de mensajería. El proyecto se estructura con la arquitectura cliente-servidor. Hace uso del protocolo TCP/IP, La comunicación entre el cliente y el servidor se da mediante sockets y el intercambio de datos con mensajes en formato JSON. Se hace uso de la programación concurrente, mediante hilos de ejecución, para atender la interacción que se puede dar entre el servidor y los múltiples clientes conectados simultáneamente a él. El servidor, escrito en el lenguaje de programación C, utiliza las bibliotecas cJSON y GLib para extender las funcionalidades nativas del mismo. El resultado se fue la implementación de un servidor capaz de soportar la carga de atender múltiples conexiones de manera concurrente y servir a las peticiones requeridas, siguiendo la especificación del protocolo de comunicación, mientras asegura la integridad de la información global de todos los usuarios.

1. Introducción

En los sistemas orientados a la mensajería en tiempo real, uno de los desafíos fundamentales radica en coordinar de forma efectiva el intercambio de información entre múltiples participantes de manera simultánea. Cuando varios usuarios intentan interactuar mediante mensajes privados, salas particulares o difusiones globales, es necesario el manejo rápido de las comunicaciones, la sincronización y la consistencia de los datos. Sin un mecanismo central de gestión, las acciones de un usuario podrían interferir con las de los demás o derivar en pérdidas de información.

El objetivo principal de este proyecto es diseñar e implementar un sistema de chat funcional que atienda estas necesidades mediante la un agente central que sirva de intermediario. El alcance del proyecto está delimitado a un escenario de escala relativamente pequeña o acotada; sin embargo, el enfoque no se limita a resolver el flujo básico de mensajes, sino que busca garantizar un nivel alto de robustez. Se prioriza que la comunicación sea fiable, de modo que el sistema responda adecuadamente ante desconexiones inesperadas, mantenga la integridad de la

información global y minimice los tiempos de espera del cliente.

2. Modelado

Habiendo establecido las necesidades del sistema y los objetivos a cumplir, modelaremos la solución analizando detalladamente los requerimientos para dividir cada problema en pequeños fragmentos más manejables para, posteriormente, diseñar componentes que los resuelvan.

2.1. Análisis

La primera división surge de manera natural por la necesidad que tenemos de servir a múltiples usuarios, por lo que tomaremos inspiración de algunas de las aplicaciones de comunicación existentes, que sabemos se sustentan bajo la arquitectura cliente-servidor. De ahí que podamos diferenciar las responsabilidades que tiene uno en comparación con el otro, sin olvidar que dependen entre sí para lograr el objetivo de comunicar.

2.2. Analizando el Servidor

El servidor es, probablemente, la parte más compleja del sistema, ya que realiza varias acciones a la vez. De manera muy general, notamos que el servidor tiene que realizar las siguientes acciones:

1. Establecer conexiones que se mantengan activas por un tiempo indeterminado.
2. Recibir peticiones, procesarlas y determinar que sean realizables.
3. Realizar de manera concurrente las operaciones que le sean solicitadas por los clientes.
4. Almacenar información global del sistema asegurando la integridad de la misma.
5. Transmitir las respuestas y notificaciones únicamente a los clientes correspondientes.
6. Exhibir un comportamiento robusto ante fallos imprevistos o errores en la comunicación.

Del primer punto, notamos que el mantener la conexión es algo que no depende enteramente del servidor. Bien puede pasar que el servidor finalice su ejecución, así como que un usuario decida desconectarse o, en casos más extremos, que la conexión entre ambos falle por motivos que escapen de nuestro control. En todos los casos anteriores, el servidor debe poder reaccionar para eliminar el registro de ese cliente y dar los avisos correspondientes.

Las acciones que el servidor puede realizar están especificadas en el protocolo de comunicación; es por eso que el punto dos está bien delimitado. Es primordial hacer que dicho protocolo se cumpla tanto al recibir como al enviar peticiones/notificaciones, ya que de esa manera aseguramos la correcta comunicación entre un cliente/servidor ajeno a este proyecto, pero que sigue la especificación del protocolo.

Atender las solicitudes de manera concurrente quiere decir que, en lugar de realizar cada acción de inicio a fin una detrás de la otra, hacemos pequeños pedazos de cada tarea para avanzar en todas. Veremos que al usar concurrencia reducimos el tiempo de espera que un cliente sufre al enviar/recibir mensajes, pero mencionaremos más adelante posibles problemas que surgen de esta estrategia.

Es de suma importancia que el servidor tenga el registro de los clientes que debe comunicar, de las salas a las que pertenecen y del modo en que puede hacerles llegar notificaciones. De la misma forma, debe asegurar que los datos registrados sean correctos, es decir, que correspondan a los clientes correctos y que reflejen las acciones que han realizado a lo largo

del tiempo. Tenemos un problema al aplicar concurrencia ya que puede ocurrir que la información sea modificada de manera incorrecta, esto ocurre debido a que la división solo aplica a las acciones a realizar y no a lo que se almacena, por lo que, si en una tarea se cambia o elimina algún dato, es casi seguro que al intentar leer ese dato, sin saber de su modificación, terminemos ocasionando errores graves en la comunicación.

Revisamos de manera indirecta los puntos cinco y seis en los párrafos anteriores, en los que hablamos de la transmisión y recepción, así como de la precauciones a tener en cuenta con ciertos errores que podemos intuir se darán.

Las acciones específicas que determina el protocolo se detallarán en el diseño, donde podremos determinar la manera de manejarlas y encapsularlas junto a las necesidades aquí descritas.

2.3. Analizando el Cliente

El cliente, al ser una pieza controlada enteramente por un usuario, nos conviene dividirlo en tres:

1. La parte más cercana al usuario, que simplificará la comunicación con el servidor, haciendo uso de comandos más sencillos de escribir y recordar.
2. La parte más cercana al servidor, que establecerá conexión y sabrá cómo comunicarse con él.
3. Un intermediario entre los dos anteriores, que sepa traducir los comandos del usuario a los mensajes que se envían al servidor y viceversa.

El primer componente, al ser una pieza enfocada a la comodidad del usuario, debe tener un lenguaje de comandos simples que ayuden a determinar rápidamente la operación a realizar. Además, el cómo presentamos las diferentes notificaciones que el servidor envía debe ser claro para quienes estén dirigidas. Exploraremos en el diseño la manera de hacer que esto ocurra.

El cliente debe poder conectarse a un servidor, así que se debe notar la necesidad de establecer y mantener una conexión activa con este último. De manera casi opuesta a lo que verá el usuario, para enviar y recibir mensajes al servidor, la segunda parte debe hacer uso enteramente del protocolo de comunicación, por lo que debe saber armar los mensajes con el formato específico y recibir las notificaciones del servidor.

Para completar el viaje de una petición de usuario, necesitamos traducir el comando que llega desde la primera parte hasta la segunda. Por tanto, la

tercera parte del cliente será un traductor entre las dos primeras. Notamos que es probablemente la más compleja, ya que tiene que determinar: la correcta escritura de comandos; enviar los datos necesarios para que la parte más cercana al servidor forme los mensajes a enviar; traducir la notificación que la segunda parte recibió y enviarla en un formato presentable a la primera.

Un detalle importante a notar es que necesitamos enviar peticiones al servidor mientras, se reciben notificaciones y se le presentan al usuario. Para ello usaremos concurrencia. Al igual que en el servidor, la ejecución concurrente debe gestionarse adecuadamente para evitar colisiones entre el flujo de lectura de notificaciones y la entrada de comandos del usuario.

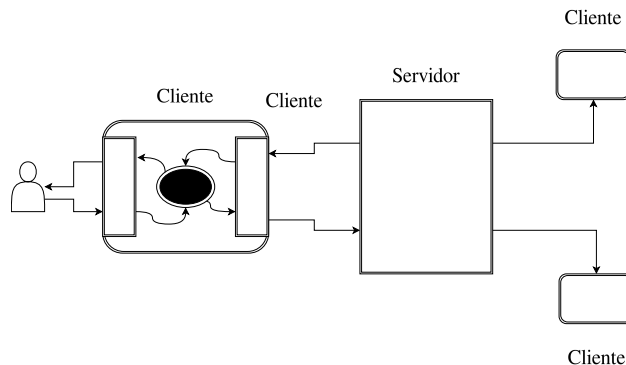


Figura 1. Diagrama de la división cliente servidor.

2.4. Diseño

Teniendo la división que vemos en la Figura 1, nos será mas sencillo identificar los componentes del sistema y de qué forma deberán comunicarse.

2.5. Diseño del servidor

Podemos agrupar las necesidades del servidor, previamente analizadas, en 'capas'. La Figura 2 ofrece una representación visual de dicha división.

2.5.1. Capa de Comunicación

Se encarga de todo lo relativo a la conexión y comunicación con clientes así como a la interpretación y estructuración de mensajes que siguen la especificación del protocolo.

Notamos entonces que podemos dividir la capa en dos componentes.

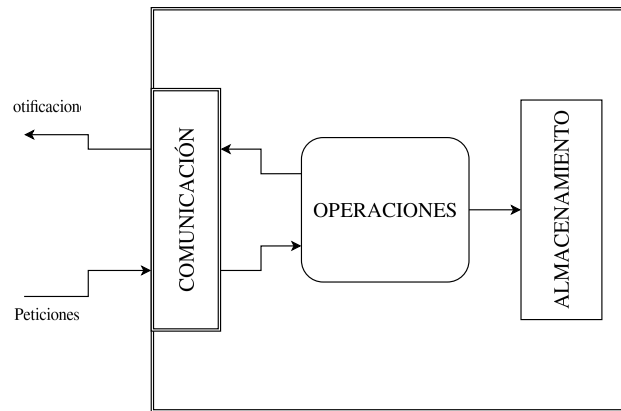


Figura 2. Diagrama de la división del servidor.

El primero siendo aquel que sabe establecer una conexión con un cliente. La conexión consta, a su vez, de dos partes: se debe identificar a un cliente para comunicarse con él y se debe poder enviar/recibir información por medio de la conexión.

Como no podemos recibir cualquier mensaje, el segundo componente deberá ser capaz de determinar si un mensaje está bien formado, esto es, que contenga el formato especificado en el protocolo aunque, como la capa de comunicación no se comunica directamente con la de almacenamiento, es imposible determinar si un mensaje es realizable pero sí podemos determinar qué tipo de petición estamos recibiendo. Al seguir en la capa más próxima a la interacción con el cliente, nos conviene que el componente también sea capaz de estructurar los mensajes de notificación.

2.5.2. Capa de Operaciones

Es la responsable de dirigir el flujo de trabajo del servidor, determinando las operaciones que se realizan y la modificación de los registros de los clientes. Además debe manejar, junto con la *capa de comunicación*, el envío de mensajes.

Sobre las operaciones a realizar, notamos que podemos tener un punto de entrada sobre el cual, una petición verificada por la *capa de comunicación*, se distribuye dependiendo el tipo de la misma o se determina como inválida dentro de los límites del protocolo.

Como se mencionaba en el análisis, todas las peticiones necesitan, en mayor o menor medida, de la información de la *capa de almacenamiento* para poder realizarse.

Dado que tenemos el problema de varias tareas realizándose a la vez, podemos tener problemas inesperados de modificación y lectura sobre los datos

que se comparten entre las distintas tareas. Una forma de solucionar dicho problema es cada que una tarea quiere acceder o modificar la información compartida tenga una manera de saber si alguna otra lo está haciendo en primer lugar y entonces esperar a que el recurso sea ‘liberado’. A esta técnica se le conoce como *exclusión mutua* y continuaremos discutiendo las partes que protegeremos de los fallos de la concurrencia en la capa de almacenamiento y en la implementación del diseño.

2.5.3. Capa de Almacenamiento

Por último en esta capa se encuentran toda las estructuras que guardan la información relativa a los clientes. Aquí se establece la manera es que se crean y eliminan dichas estructuras.

Es primordial que guardemos a cada cliente que haya establecido conexión con el servidor y se haya identificado apropiadamente.

Un primer acercamiento a esta necesidad nos deja en claro que cada cliente tiene datos que necesitamos mantener accesibles: la identificación de un cliente para poder enviar información mediante la conexión que establecimos con él; un identificador conocido por los demás clientes conectados; y, siguiendo el protocolo, su estado en el servidor (ACTIVE, BUSY, AWAY).

Se tienen que almacenar los clientes en una estructura que nos permita agregar a cuántos usuarios decidan conectarse al servidor y eliminarlos cuando se desconecten. Además, en el protocolo se especifica la posibilidad de crear salas privadas, dónde solo algunos usuarios, del total de conectados, pueden enviar mensajes que solo recibirán ellos. Además pueden existir múltiples salas en el servidor. Por ello también se contempla la creación de una estructura que guarde salas (dónde sobra decir que pueden agregar nuevas salas y eliminarlas cuando dejen de tener usuarios) dónde cada sala es, a su vez, una estructura que guarda usuarios dentro de la y aquellos invitados a estar dentro de ella.

Visto de forma global, podemos crear una estructura que almacena a los clientes de todo el servidor y las salas que en él se crearon cada uno con sus respectivos clientes.

Sin dejar a un lado que debemos proteger a toda esta estructura global de las múltiples modificaciones que pueden resultar de la concurrencia. Discutiremos de forma concreta qué hacemos para lograr este objetivo en los detalles de la implementación.

3. Implementación

Se opta por el lenguaje de programación C para implementar el servidor ya que este, a pesar de las pocas facilidades que ofrece al programador, resulta sumamente enriquecedor para entender conceptos fundamentales como el manejo de memoria dinámica.

Sin embargo el objetivo no es reinventar la rueda. Por este motivo también se usan tecnologías y bibliotecas de terceros que ayudan enormemente al desarrollo de un proyecto de esta magnitud.

Empezando por el sistema de construcción que ayuda a mantener centralizado todo lo relativo a la compilación, manejo de dependencias y organización general del proyecto. En el caso de la implementación presentada se usa el sistema de construcción: *Meson*, el cual maneja todo lo necesario para la instalación de las dependencias necesarias así como de la compilación del proyecto entero. Resulta bastante intuitivo una vez que se explora con cierto detalle las funcionalidades básicas que posee.

Hoy en un día, usar un manejador de versiones para llevar el progreso de cualquier proyecto, es un estándar. Con diferencia el más popular es **git** y es el que se usa para llevar un control de los cambios y modificaciones en el desarrollo del sistema.

Las bibliotecas usadas para el servidor son **GLib** y **cJSON**. La primera implementa y extiende muchas de las funcionalidades de la biblioteca estándar de C, en particular se usa la implementación de *GLib* de diccionarios (GHashTable) para cumplir con el diseño de la *Capa de Almacenamiento* del servidor que se modela en la Figura 3

El diseño concurrente se implementa usando los hilos de ejecución propios de la biblioteca *GLib*, los mismos se encuentran en el punto de entrada del servidor y se crea uno nuevo al establecer la conexión con un cliente. Para mantener la consistencia de la información almacenada se implementó la exclusión mutua en la estructura *ContextoServidor*, haciendo uso de la estructura *GMutex*.

Para establecer conexiones y cumplir con el manejo correcto del protocolo de comunicación, siguiendo el diseño de la *Capa de Comunicación*, se usa la API de POSIX para sockets en C, que proveen de todo lo necesario para establecer conexiones en redes locales y comunicarse haciendo uso del protocolo TCP/IP, ideal para el tipo de sistema que realizamos.

Un detalle de implementación es que debido a lo largo y complicado que puede ser el código necesario para llevar a cabo las conexiones haciendo uso de sockets se optó por envolverla en un solo componente, *socket_util*, que establece las funciones para iniciar conexiones, esperar un conexión y enviar información

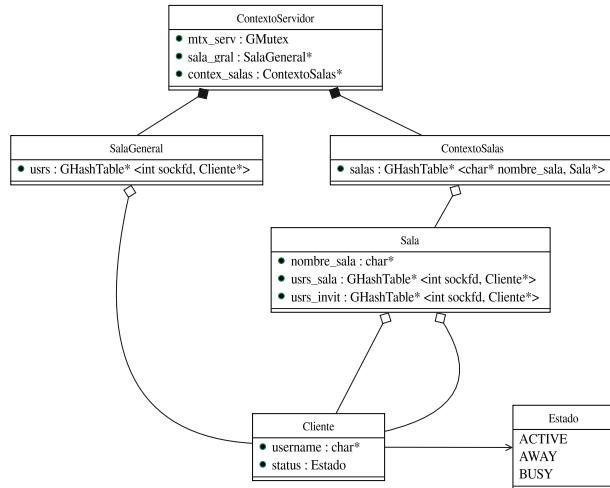


Figura 3. Diagrama de las estructuras del servidor siguiendo el diseño propuesto.

a través de una.

Se crea además un componente, *protocolo*, que define enumeraciones y mapeos a cadenas para poder identificar rápidamente los tipos de mensajes, operaciones, resultados y estados que se pueden enviar en un JSON. A su vez, usando los patrones de diseño **DTO** (Data Transfer Object) y *Factory* se crea una estructura *Campos* que se pasa como una **Literal Compuesta** (Compound Literals) en C a la función *fabrica_espuesta* que usa la biblioteca *cJSON* para traducir y empaquetar la información necesaria en la comunicación.

En la *Capa de Operaciones*, tenemos a los componentes *distribuidor* y *manejador* que se encargan, dependiendo el tipo de mensaje, de realizar las peticiones necesarias y enviar las notificaciones correspondientes en cada caso.

4. Conclusión

Se logró abstraer con éxito la complejidad del desarrollo de un servidor en C mediante el uso de tipos abstractos de datos, patrones de diseño y la biblioteca GLib, alcanzando un servidor de chat multihilo estable, capaz de gestionar múltiples salas y usuarios de forma concurrente.

Fue un gran proyecto que ante todo marca un antes y un después en lo que creí que me tomaría años aprender. A la vez resulta ser uno de los que más satisfacción me deja a pesar de las complicaciones y contratiempos que hicieron que no pudiera termi-

narlo como me gustaría.

Fue abrumador balancear el análisis y el diseño de cosas que no terminaba de entender por qué eran necesarias o cómo funcionaban. Y, una vez puesto en práctica, los errores referentes a las bibliotecas desconocidas resultaban abrumadores.

Me llevo mucha experiencia y ganas de continuar haciendo proyecto que no me veía haciendo antes de entrar a la Licenciatura.